Week 7. eBPF Deep Dive & Hubble

1. eBPF 동작 원리

1-1. 구성 요소

구성 요소	역할	Cilium에서 보이는 예
Program	hook에서 실행되는 코드	정책 판정, 서비스 로드밸런싱
Hook	프로그램이 붙는 커널 이벤트 지점	XDP, tc, cgroup
Map	커널 안 key-value 저장소 (프로그램 ↔ 유저스페이스 공유)	ipcache, 서비스, 정책, conntrack
Verifier	로드 시 안전성 정적 검증	실패하면 로드 거부
JIT	바이트코드 → 네이티브 기계어	실행 속도 확보
Helper	프로그램이 커널 기능을 쓰는 안정된 API	map 조회, 패킷 조작

- 프로그램은 **임의의 커널 함수를 호출할 수 없고**, 정해진 helper API만 사용 → 커널 버전이 바뀌어도 이식성 유지
- 이벤트 기반: 커널이나 애플리케이션이 해당 hook 지점을 지날 때마다 실행

1-2. 로드부터 실행까지

1. 제한된 C(pseudo-C)로 작성 → clang/LLVM으로 **eBPF 바이트코드** 컴파일
2. `bpf()` 시스템 호출로 커널에 로드 (보통 라이브러리 경유)
3. **Verifier**가 정적 분석 → 통과해야만 다음 단계로 진행
4. **JIT**가 네이티브 기계어로 변환
5. 요청한 **hook에 attach**
6. 이벤트(패킷 수신, syscall 등)가 올 때마다 실행

- 로드하는 쪽은 root 또는 `CAP_BPF` 권한이 필요

1-3. Verifier가 확인하는 것

검사 항목	내용
종료 보장	프로그램은 반드시 끝나야 함 → 종료를 증명할 수 있는 **bounded loop**만 허용
메모리 안전	범위를 벗어난 접근, 초기화 안 된 값 사용 금지
레지스터 추적	각 레지스터의 타입과 값의 범위를 경로별로 추적
스택	쓰기 전에 읽을 수 없음
포인터	map 조회 결과는 NULL 검사 후에만 사용
helper 호출	프로그램 타입별로 허용된 helper만, 인자 조건에 맞게 호출
크기/복잡도	프로그램 크기와 분석 복잡도에 상한

- verifier는 **안전(커널이 죽지 않음)** 검사이지, 프로그램이 하는 일이 옳은지 보는 건 아님
- 실패하면 로드 자체가 거부 → 커널 크래시를 막는 안전장치

1-4. 알아 둘 한계 (공식 문서)

- 레지스터 11개(`r0` ~ `r10`), **스택 512바이트**
- 한 프로그램이 직접 접근 가능한 map: 최대 64개
- 프로그램 크기: 원래 4096 명령어 → 커널 5.1 이후 100만 명령어로 상향
- **Tail call**: 다른 프로그램으로 제어를 넘기고 돌아오지 않음 (`execve()`와 비슷) → 최대 중첩 깊이 33

2. BPF Map & Cilium Datapath

2-1. BPF Map의 역할

- 커널과 유저스페이스 사이에서 **데이터를 공유하는 범용 저장소**
- 프로그램은 helper로 읽고 쓰고, 유저스페이스(agent, bpftool)는 `bpf()` 시스템 호출로 접근
  - `BPF_MAP_CREATE`, `BPF_MAP_LOOKUP_ELEM`, `BPF_MAP_UPDATE_ELEM`, `BPF_MAP_DELETE_ELEM`
- 주요 타입: hash, array, LRU hash, **LPM trie**(longest prefix match → CIDR/IP 조회에 적합), per-CPU 변형, prog array(tail call용), ringbuf 등
- 프로그램끼리도 map으로 상태를 공유 (프로그램은 서로 직접 상태를 공유하지 않음)

- 로더 프로세스가 종료돼도 프로그램과 map이 유지되도록 **bpffs에 pin** 가능 (`BPF_OBJ_PIN` / `BPF_OBJ_GET`)

역할 분담

```
cilium-agent (유저스페이스)  →  map에 쓰기 (서비스, identity, 정책 …)
eBPF Program (커널)          →  map을 읽어 패킷 판정
```

- **프로그램은 판정 로직만, 상태는 map이 보관**
  → 정책이 바뀌어도 프로그램을 다시 올릴 필요 없이 map 내용만 갱신

2-2. Cilium이 쓰는 주요 Map

종류	담고 있는 것	조회 명령
IP cache (ipcache)	IP/CIDR → **security identity** 매핑 (로컬·원격 endpoint 모두)	`cilium-dbg bpf ipcache list`
Endpoint	노드의 로컬 endpoint와 호스트 IP	`cilium-dbg bpf endpoint list`
Service LB	서비스 frontend → backend 매핑 (ClusterIP, NodePort)	`cilium-dbg bpf lb list`
Policy	endpoint별 (identity, 포트, 프로토콜) 허용 규칙	`cilium-dbg bpf policy get --all`
Connection Tracking	진행 중인 연결 상태	`cilium-dbg bpf ct list global`
NAT	NAT 매핑	`cilium-dbg bpf nat list`

2-3. 기본 용량

Map	범위	기본 상한
Connection Tracking	노드	TCP 512k / UDP 256k
NAT	노드	512k
IP cache	노드	512k
Endpoints	노드	64k
Service Load Balancer	노드	64k
Service Backends	노드	64k
Policy	endpoint	16k

- LB map 한 서비스당 엔트리 수 ≈ (서비스의 endpoint 수) × (서비스의 port/protocol 수)
- 조정 방법: agent 플래그(`--bpf-ct-global-tcp-max`, `--bpf-lb-map-max` 등) 또는 `--bpf-map-dynamic-size-ratio` (전체 메모리 대비 비율)
- NAT 크기는 CT 크기 합의 2/3를 넘지 않게 설정해야 함
- **LB map 크기를 바꾸고 재시작하면 map을 다시 채우는 동안 연결이 끊김**

2-4. Program이 Map을 참조해 패킷을 처리하는 과정

단계	하는 일	쓰는 Map
① 서비스 주소 변환	목적지가 Service면 frontend를 찾아 backend Pod IP로 선택	Service LB
② identity 확인	목적지 IP가 어떤 identity에 속하는지 조회	ipcache
③ 정책 판정	출발지·목적지 identity, 포트, 프로토콜이 허용 규칙에 있는지	Policy
④ 연결 상태 기록	허용된 연결을 추적해 이후 패킷과 응답 처리	Connection Tracking
⑤ 결과	허용 → 전달 / 불허 → drop / L7 규칙 → Envoy로 redirect	-

- 정책의 키는 **IP가 아니라 identity** → Pod가 재생성되어 IP가 바뀌어도 ipcache 매핑만 갱신되면 정책은 그대로

3. Hubble & Troubleshooting

3-1. eBPF Event → Hubble Flow

```
eBPF Program (drop / trace / policy-verdict 이벤트 발생)
      ↓
Cilium monitor (이벤트 수신)
      ↓
Hubble server (agent에 내장, monitor에 listener 등록)
      ↓
ring buffer (agent 메모리, 크기 설정 가능, 가득 차면 오래된 것부터 덮어씀)
      ↓
Observer API (GetFlows)
      ↓
Hubble CLI (노드 로컬 unix socket) / Hubble Relay (클러스터 전체) / Hubble UI
```

- 판정하는 프로그램이 곧 **이벤트를 내보내는 지점** → 실행과 관찰이 같은 데이터패스에서 발생
- `cilium-dbg monitor`도 같은 이벤트를 보는 도구 (`--type`: drop, trace, policy-verdict, l7, capture, debug ...)
- Hubble server는 기본적으로 **노드 단위**로 동작 → 클러스터 전체를 보려면 **Hubble Relay**가 모든 노드 peer에 연결해 통합

3-2. Flow에 담기는 정보

필드	의미
`verdict`	이 이벤트의 판정 결과
`source` / `destination`	endpoint 정보 (identity, labels, namespace, pod_name)
`IP`, `l4`	L3/L4 정보 (주소, 프로토콜, 포트)
`traffic_direction`	ingress / egress
`trace_observation_point`	이벤트가 관측된 지점 (trace 이벤트에서만 의미 있음)
`is_reply`	응답 방향 패킷인지
`drop_reason_desc`	drop 이유 (verdict가 DROPPED일 때만)
`event_type`	원본 Cilium 이벤트 종류

3-3. Verdict 의미

Verdict	의미
FORWARDED	trace 지점이 패킷/연결을 **다음 처리 단계로 넘김**
DROPPED	패킷/연결이 drop됨
ERROR	처리 중 오류가 난 이벤트
AUDIT	**audit 모드**에서 정책이 막았을 트래픽
REDIRECTED	L7 proxy로 redirect됨
TRACED	trace 지점에서 관측됨
TRANSLATED	주소가 변환됨

대표적인 Trace 관측 지점

지점	의미
FROM_ENDPOINT / TO_ENDPOINT	컨테이너에서 받음 / 컨테이너로 보냄
FROM_NETWORK / TO_NETWORK	물리(native) 디바이스에서 받음 / 보냄

지점	의미
FROM_OVERLAY / TO_OVERLAY	터널 디바이스에서 받음 / 보냄
FROM_HOST / TO_HOST	호스트 네임스페이스 쪽
FROM_STACK / TO_STACK	호스트 리눅스 네트워크 스택 쪽
FROM_PROXY / TO_PROXY	L7 프록시(Envoy) 쪽

- FORWARDED가 보이는 **지점**을 따라가면 패킷이 어디까지 갔는지 짐작 가능 (예: TO_ENDPOINT까지 갔는데 응답이 없으면 목적지 앱 쪽 문제)

3-4. 통신 문제 추적 순서

1. **Hubble 연결 확인**: `hubble status`
2. **끊긴 흐름 찾기**: `hubble observe --pod <대상> --verdict DROPPED`
3. **drop 이유 확인**: `drop_reason_desc` (정책 때문이면 Policy denied)
4. **정책/라벨 확인**: 출발지·목적지 라벨, 적용된 정책
5. **identity 확인**: `cilium-dbg ip list | grep <IP>`로 IP가 기대한 identity를 갖는지
6. **정책 수정 후 재검증**: 다시 DROPPED가 사라지고 FORWARDED가 나오는지 확인
7. 엄격하게 바꾸기 전에는 **audit 모드**로 AUDIT 이벤트부터 확인

증상	Hubble에서 보이는 것	의미 / 다음 확인
연결이 안 됨	DROPPED + Policy denied	허용 규칙 없음 → 라벨, 정책 확인
연결은 되는데 응답이 없음	FORWARDED만 있고 reply 없음	목적지 앱, 응답 경로, 반대 방향 flow 확인
Service로 접속이 이상함	TRANSLATED	어느 backend로 변환됐는지 확인
정책 도입 전 점검	AUDIT	막혔을 트래픽을 미리 파악
L7 규칙 트래픽	REDIRECTED	Envoy 경유, L7 flow로 확인
flow가 아예 없음	(없음)	다른 노드에서 발생(Relay 사용), ring buffer 덮어쓰기, Cilium 데이터패스를 안 거치는 경로

4. Hands-on

4-1. bpftool로 프로그램 / Map 확인

```
1939: sched_cls  name tail_handle_ipv4_cont  tag 1a5beddea66e91c2  gpl
1940: sched_cls  name tail_handle_arp  tag 210da590646efc92  gpl
1941: sched_cls  name tail_ipv4_ct_egress  tag ed73af9a5bfb3dc0  gpl
1943: sched_cls  name tail_no_service_ipv4  tag 524138bd9681366f  gpl
1944: sched_cls  name cil_lxc_policy  tag 9e007e2259856a49  gpl
1945: sched_cls  name tail_ipv4_policy  tag c41ef863ac6002c7  gpl
1946: sched_cls  name cil_lxc_policy_egress  tag 691ef55febf78432  gpl
1947: sched_cls  name cil_from_container  tag b1520d937090b4d6  gpl
1948: sched_cls  name tail_drop_notify  tag d35c1d928a577a35  gpl
1949: sched_cls  name tail_ipv4_to_endpoint  tag 424019b602880193  gpl
1950: sched_cls  name tail_handle_ipv4  tag bba178f4f818ab43  gpl
1951: sched_cls  name tail_ipv4_ct_ingress  tag 96872acfd965f352  gpl
1952: sched_cls  name tail_drop_notify  tag d35c1d928a577a35  gpl
1953: sched_cls  name tail_ipv4_ct_ingress  tag 96872acfd965f352  gpl
1955: sched_cls  name cil_lxc_policy  tag 9e007e2259856a49  gpl
1956: sched_cls  name tail_ipv4_policy  tag c41ef863ac6002c7  gpl
1958: sched_cls  name tail_handle_ipv4_cont  tag 1a5beddea66e91c2  gpl
1959: sched_cls  name tail_ipv4_to_endpoint  tag 424019b602880193  gpl
1960: sched_cls  name cil_lxc_policy_egress  tag 691ef55febf78432  gpl
```

- Cilium 프로그램 이름 목록 (`cil_from_container`, `tail_ipv4_policy` 등)
  `sched_cls` 186개 중 132개가 `tail_*` 이름 → tail call로 쪼개진 구조

```
603: lru_percpu_hash  name cilium_policyst  flags 0x2
637: lru_percpu_hash  name cilium_policyst  flags 0x2
670: lru_percpu_hash  name cilium_policyst  flags 0x2
699: lpm_trie  name cilium_policy_v  flags 0x81
700: lpm_trie  name cilium_policy_v  flags 0x81
707: lpm_trie  name cilium_policy_v  flags 0x81
708: lpm_trie  name cilium_policy_v  flags 0x81
709: lpm_trie  name cilium_policy_v  flags 0x81
710: lpm_trie  name cilium_policy_v  flags 0x81
711: lpm_trie  name cilium_policy_v  flags 0x81
718: lpm_trie  name cilium_policy_v  flags 0x81
719: lpm_trie  name cilium_policy_v  flags 0x81
985: lpm_trie  name cilium_policy_v  flags 0x81
992: lpm_trie  name cilium_policy_v  flags 0x81
1004: lpm_trie  name cilium_policy_v  flags 0x81
1020: lpm_trie  name cilium_policy_v  flags 0x81
1037: lpm_trie  name cilium_policy_v  flags 0x81
```

- `cilium_policy_v` 14개(policy map은 endpoint별로 하나씩이라 개수가 많음),
  `cilium_policyst` 3개

```
608: hash  name cilium_lb4_serv  flags 0x81
642: hash  name cilium_lb4_serv  flags 0x81
675: hash  name cilium_lb4_serv  flags 0x81
596: lpm_trie  name cilium_ipcache_  flags 0x81
630: lpm_trie  name cilium_ipcache_  flags 0x81
660: lpm_trie  name cilium_ipcache_  flags 0x81
```

- `bpftool map show` 중 `cilium_lb4_serv` 3개, `cilium_ipcache_` 3개 (노드별 map이라 노드 수 3과 일치)

4-2. Cilium BPF Map 확인

```
10.244.1.218/32     identity=25119 encryptkey=0 tunnelendpoint=0.0.0.0 flags=<no
10.244.3.64/32      identity=3583 encryptkey=0 tunnelendpoint=172.18.0.3 flags=h
10.244.1.66/32      identity=10479 encryptkey=0 tunnelendpoint=0.0.0.0 flags=<no
```

- server / client / intruder IP의 identity (25119 / 3583 / 10479), client의 `tunnelendpoint=172.18.0.3`

```
Endpoint ID: 26
Path: /sys/fs/bpf/tc/globals/cilium_policy_v3_00026

POLICY   DIRECTION   LABELS (source:key[=value])       PORT/PROTO   PROXY PORT
 AUTH TYPE   BYTES   PACKETS   PREFIX   COOKIE
Allow    Ingress     ANY                               ANY          NONE
 disabled    -       -         0        0
Allow    Ingress     reserved:aggregate-cluster        ANY          NONE
 disabled    1080    14        0        0
Allow    Ingress     reserved:aggregate-cluster-mesh   ANY          NONE
 disabled    -       -         0        0
Allow    Ingress     reserved:aggregate-world          ANY          NONE
 disabled    -       -         0        0
Allow    Ingress     reserved:aggregate-remote-node    ANY          NONE
 disabled    -       -         0        0
Allow    Egress      ANY                               ANY          NONE
 disabled    -       -         0        0
Allow    Egress      reserved:aggregate-cluster        ANY          NONE
 disabled    -       -         0        0
Allow    Egress      reserved:aggregate-cluster-mesh   ANY          NONE
 disabled    -       -         0        0
Allow    Egress      reserved:aggregate-world          ANY          NONE
 disabled    -       -         0        0
Allow    Egress      reserved:aggregate-remote-node    ANY          NONE
 disabled    -       -         0        0
```

- `Endpoint ID: 26` (server) 블록, 정책 적용 **전** 상태

4-3. NetworkPolicy 적용 전/후 Flow 비교

```
NAME       READY   STATUS    RESTARTS   AGE   IP             NODE
  NOMINATED NODE   READINESS GATES
client     1/1     Running   0          32s   10.244.3.64    cilium-lab-worker2
  <none>           <none>
intruder   1/1     Running   0          27s   10.244.1.66    cilium-lab-worker
  <none>           <none>
server     1/1     Running   0          40s   10.244.1.218   cilium-lab-worker
  <none>           <none>
```

- client(worker2) / server·intruder(worker)의 IP와 node

```
Oct 15 10:46:23.048: demo/client:39236 (ID:3583) -> demo/server:80 (ID:25119) to
-overlay FORWARDED (TCP Flags: SYN)
Oct 15 10:46:23.048: demo/client:39236 (ID:3583) -> demo/server:80 (ID:25119) to
-endpoint FORWARDED (TCP Flags: SYN)
Oct 15 10:46:23.048: demo/client:39236 (ID:3583) <- demo/server:80 (ID:25119) to
-overlay FORWARDED (TCP Flags: SYN, ACK)
Oct 15 10:46:23.048: demo/client:39236 (ID:3583) <- demo/server:80 (ID:25119) to
-endpoint FORWARDED (TCP Flags: SYN, ACK)
Oct 15 10:46:23.048: demo/client:39236 (ID:3583) -> demo/server:80 (ID:25119) to
-overlay FORWARDED (TCP Flags: ACK)
Oct 15 10:46:23.048: demo/client:39236 (ID:3583) -> demo/server:80 (ID:25119) to
-endpoint FORWARDED (TCP Flags: ACK)
Oct 15 10:46:23.049: demo/client:39236 (ID:3583) -> demo/server:80 (ID:25119) to
-overlay FORWARDED (TCP Flags: ACK, PSH)
Oct 15 10:46:23.049: demo/client:39236 (ID:3583) -> demo/server:80 (ID:25119) to
-endpoint FORWARDED (TCP Flags: ACK, PSH)
Oct 15 10:46:23.049: demo/client:39236 (ID:3583) <- demo/server:80 (ID:25119) to
-overlay FORWARDED (TCP Flags: ACK, PSH)
Oct 15 10:46:23.049: demo/client:39236 (ID:3583) <- demo/server:80 (ID:25119) to
-endpoint FORWARDED (TCP Flags: ACK, PSH)
Oct 15 10:46:23.050: demo/client:39236 (ID:3583) -> demo/server:80 (ID:25119) to
-overlay FORWARDED (TCP Flags: ACK, FIN)
Oct 15 10:46:23.050: demo/client:39236 (ID:3583) -> demo/server:80 (ID:25119) to
-endpoint FORWARDED (TCP Flags: ACK, FIN)
Oct 15 10:46:23.051: demo/client:39236 (ID:3583) <- demo/server:80 (ID:25119) to
-overlay FORWARDED (TCP Flags: ACK, FIN)
Oct 15 10:46:23.051: demo/client:39236 (ID:3583) <- demo/server:80 (ID:25119) to
-endpoint FORWARDED (TCP Flags: ACK, FIN)
Oct 15 10:46:23.051: demo/client:39236 (ID:3583) -> demo/server:80 (ID:25119) to
-overlay FORWARDED (TCP Flags: ACK)
```

- client → server `FORWARDED` (`to-overlay` / `to-endpoint`) 정책 **전**

```
ENDPOINT   POLICY (ingress)   POLICY (egress)   IDENTITY   LABELS (source:key[=v
alue])                                                        IPv6   IPv4
     STATUS
          ENFORCEMENT        ENFORCEMENT


26         Disabled           Disabled          25119      k8s:app=server
                                                                        10.244.1.2
18   ready
                                                           k8s:io.cilium.k8s.nam
espace.labels.kubernetes.io/metadata.name=demo

                                                           k8s:io.cilium.k8s.pol
icy.cluster=default

                                                           k8s:io.cilium.k8s.pol
icy.serviceaccount=default

                                                           k8s:io.kubernetes.pod
.namespace=demo
```

- server(endpoint 26)의 ingress enforcement `Disabled` (정책 **전**)

```
2026-10-15  19:50.35  /home/mobaxterm/dl  kubectl -n demo exec clie
nt -- curl -s -m 3 -o /dev/null -w '%{http_code}\n' server
200

2026-10-15  19:50.41  /home/mobaxterm/dl  kubectl -n demo exec intr
uder -- curl -s -m 3 -o /dev/null -w '%{http_code}\n' server
000
command terminated with exit code 28
```

- client `200`, intruder `000` + `exit code 28` 정책 **후**

```
Oct 15 10:50:46.193: demo/intruder:46824 (ID:10479) <> demo/server:80 (ID:25119)
 policy-verdict:none INGRESS DENIED (TCP Flags: SYN)
Oct 15 10:50:46.194: demo/intruder:46824 (ID:10479) <> demo/server:80 (ID:25119)
 Policy denied DROPPED (TCP Flags: SYN)
Oct 15 10:50:47.195: demo/intruder:46824 (ID:10479) <> demo/server:80 (ID:25119)
 policy-verdict:none INGRESS DENIED (TCP Flags: SYN)
Oct 15 10:50:47.195: demo/intruder:46824 (ID:10479) <> demo/server:80 (ID:25119)
 Policy denied DROPPED (TCP Flags: SYN)
Oct 15 10:50:48.219: demo/intruder:46824 (ID:10479) <> demo/server:80 (ID:25119)
 policy-verdict:none INGRESS DENIED (TCP Flags: SYN)
Oct 15 10:50:48.219: demo/intruder:46824 (ID:10479) <> demo/server:80 (ID:25119)
 Policy denied DROPPED (TCP Flags: SYN)
```

- `policy-verdict:none INGRESS DENIED`, `Policy denied DROPPED`
  - SYN이 1초 간격으로 재전송되며 계속 DROPPED

```
ENDPOINT   POLICY (ingress)   POLICY (egress)   IDENTITY   LABELS (source:key[=v
alue])                                                        IPv6   IPv4
     STATUS
           ENFORCEMENT        ENFORCEMENT

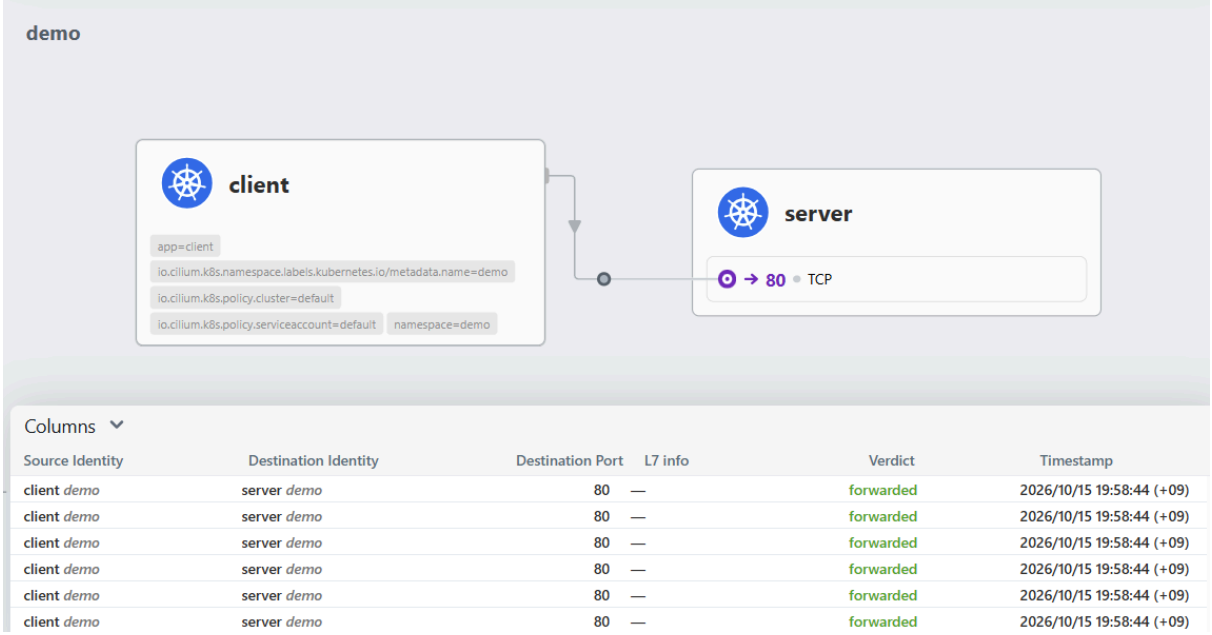

26         Enabled            Disabled          25119      k8s:app=server
                                                                        10.244.1.2
18   ready
                                                           k8s:io.cilium.k8s.nam
espace.labels.kubernetes.io/metadata.name=demo

                                                           k8s:io.cilium.k8s.pol
icy.cluster=default

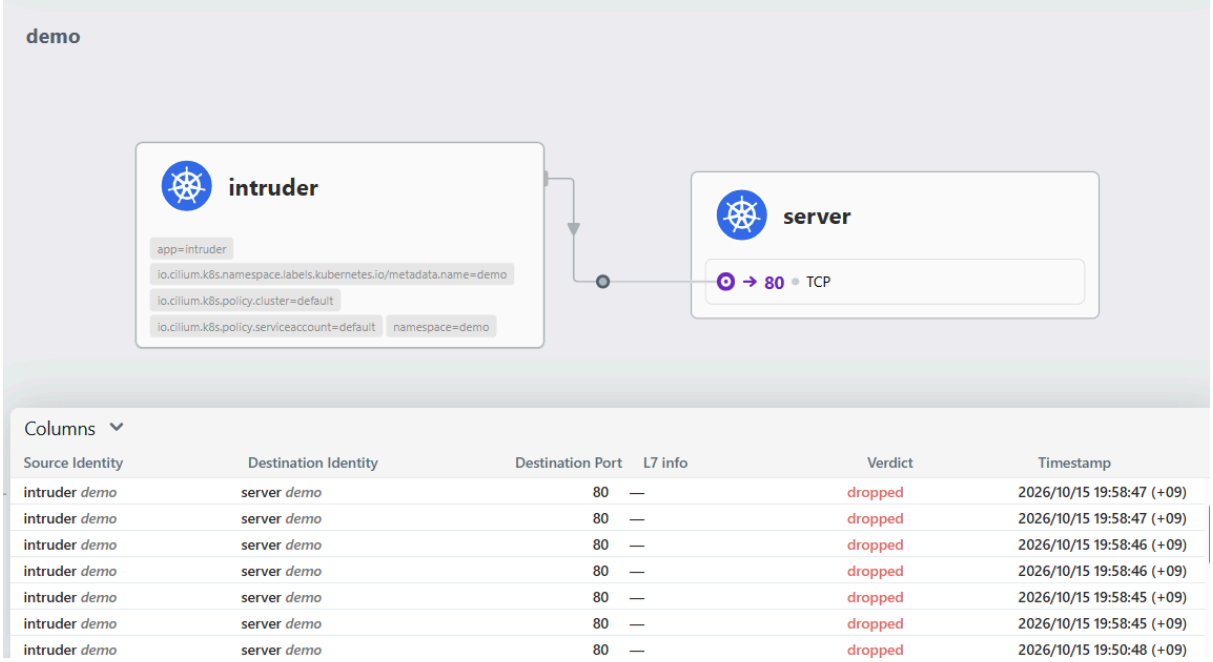
                                                           k8s:io.cilium.k8s.pol
icy.serviceaccount=default

                                                           k8s:io.kubernetes.pod
.namespace=demo
```

- server(endpoint 26)의 ingress enforcement `Enabled` (정책 **후)**



- client → server `forwarded`



- intruder → server `dropped`

결론

> eBPF 프로그램은 hook에서 **map을 조회해 판정**하고, 그 판정 이벤트를 Hubble이 **flow로 노출**한다

Q1. eBPF Program이 어떻게 Kernel에서 실행되는가

- 작성(제한된 C) → 바이트코드 컴파일 → `bpf()`로 커널 로드
- **Verifier**가 안전성 검증(종료 보장, 메모리 접근, 레지스터·포인터 추적) → 통과해야 로드
- **JIT**가 기계어로 변환 → **hook에 attach**
- hook 이벤트(패킷 수신, syscall 등)가 올 때마다 실행
- 커널 함수는 helper API로만 호출, 상태는 **map**으로 보관, 다른 프로그램으로는 tail call(최대 깊이 33)

Q2. Cilium이 BPF Map을 이용해 네트워크 상태를 관리하는 방식

- agent(유저스페이스)가 map을 **채우고**, 데이터패스 프로그램(커널)은 map을 **조회만** 함

- 주요 map: ipcache(IP→identity), Service LB(frontend→backend), Policy(identity·포트·프로토콜), Connection Tracking, NAT
- 정책의 키는 IP가 아니라 identity → Pod IP가 바뀌어도 ipcache만 갱신하면 정책 유지
- 용량 상한(예: CT 512k, LB 64k)이 있어 규모에 맞춰 조정 필요, LB map 변경은 연결 끊김을 동반

Q3. eBPF Datapath에서 발생한 이벤트가 Hubble Flow로 연결되는 과정

- 정책을 판정하는 프로그램이 곧 이벤트(drop, trace, policy-verdict)를 내보내는 지점
- Cilium monitor가 이벤트를 받고, agent에 내장된 Hubble server가 listener로 받아 ring buffer에 적재
- Observer API(`GetFlows`)로 노출 → CLI는 노드 로컬, Relay는 클러스터 전체
- flow에는 identity, 라벨, verdict, 방향, 관측 지점, drop 이유가 담김

Q4. Hubble을 이용해 통신 성공/실패 원인을 추적하는 방법

- 먼저 `--verdict DROPPED`로 끊긴 흐름을 찾고 `drop_reason_desc`로 이유 확인
- 정책 때문이면 라벨, identity(`cilium-dbg ip list`), 정책 규칙 순으로 확인 후 수정, 재검증
- FORWARDED인데 응답이 없으면 trace 관측 지점과 reply 방향 flow로 어디까지 갔는지 확인
- 정책 도입 전에는 audit 모드의 AUDIT 이벤트로 미리 점검
- flow가 안 보일 때는 다른 노드(Relay), ring buffer 덮어쓰기, 데이터패스를 안 거치는 경로(보조 NIC 등)를 의심

질문

문서를 보니 CT는 TCP 512k, LB는 64k 같은 기본 상한이 있고 LB map 크기를 바꾸고 재시작하면 연결이 끊긴다고 되어 있음. 클러스터가 커지면 map이 가득 찰 수도 있을 것 같은데, 가득 차면 실제로 어떤 증상이 나타나는지(신규 연결만 실패하는지 등)와 미리 감지하려면 어떤 지표를 보는지 궁금함.